

Take-Assessment

Grocery Booking System · Laravel Backend Track

ROLE	EXPERIENCE LEVEL	Deadline
Full Stack Engineer (PHP/Laravel)	2–4 years, Laravel	3rd October 2026

Build a small but complete **Grocery Booking System** in Laravel where an **Admin** manages inventory and a **User** browses items and places bookings. We're evaluating your architecture and judgment, not just whether the CRUD works.

1 Roles

- Admin — manages the product catalogue and stock
- User — browses items and places bookings

2 Authentication & Access Control

- JWT-based authentication: register, login, logout, token refresh (tymon/jwt-auth recommended)
- Role-based access enforced at the middleware/guard level — not inside individual controller methods

3 Architecture Requirement

- Repository Design Pattern: Interface + Eloquent implementation, bound via a Service Provider
 - A Service layer between Controller and Repository is encouraged, though not mandatory

4 API Endpoints

Admin can:

- ✓ Add new grocery items to the system
- ✓ View existing grocery items
- ✓ Remove grocery items from the system
- ✓ Update item details (name, price, etc.)
- ✓ Manage inventory / stock levels

User can:

- ✓ View the list of available grocery items
- ✓ Book multiple grocery items in a single order
- ✓ View their own order history

5 Frontend — Blade + AJAX

- Build the user-facing browsing and booking flow using Blade views
- At least one interaction (e.g. add-to-order, live stock check) must run over AJAX with no full page reload

6 Database

- Any relational database is fine — MySQL preferred
- Design proper relationships across Users, Roles, Grocery Items, Orders, and Order Items
 - Stock deduction on order placement should be transaction-safe (no overselling under concurrent requests)

7 Bonus Points

- Containerize the app with Docker (app + database via docker-compose)
- Localization: support at least English and Bangla on user-facing Blade views

8

Submission Checklist

- ✓ GitHub repository link shared via your submission email
- ✓ README with setup steps, endpoint list, and a short note on your architectural decisions
- ✓ Submitted within the deadline

We'd rather you ask a clarifying question than guess — reach out any time during the deadline.